# CV-Tools — Référence des filtres

Chaque filtre, paramètre et mise en garde en un seul document

# Référence des filtres

Chaque filtre est une simple fonction qui prend une image en premier argument et renvoie une nouvelle image. Les noms de registre (colonne `name`) sont ceux qui apparaissent dans les préréglages JSON et les rapports.

**Les images sont en RVB.** `ImageLoader` convertit au chargement et `save_image` reconvertit à l'enregistrement : chaque filtre reçoit donc le canal 0 comme rouge — et non le BGR que renvoie `cv2.imread`. Cela compte pour le paramètre `channel` et pour tout traitement sensible à la couleur ; une opération de luminance comme CLAHE ne voit aucune différence, ce qui rend précisément l'erreur facile à manquer.

Les mesures qui renvoient des chiffres plutôt qu'une image — bruit, ELA, copier-déplacer, compression, fantôme JPEG et métadonnées — ne sont pas des étapes de chaîne et sont regroupées à part, sous Rapports d'analyse.

| Nom dans le registre | Fonction | Module | Sprint |
|---|---|---|---|
| `clahe` | `apply_clahe` | `cv_tools.filters.clahe` | 1 |
| `contrast_brightness` | `adjust_contrast_brightne` | `cv_tools.filters.contrast_brightness` | 1 |
| `auto_contrast` | `auto_contrast` | `cv_tools.filters.contrast_brightness` | 1 |
| `levels` | `adjust_levels` | `cv_tools.filters.levels` | 1 |
| `auto_levels` | `auto_levels` | `cv_tools.filters.levels` | 1 |
| `histeq` | `histogram_equalization` | `cv_tools.filters.histogram_equalization` | 1 |
| `roi_crop` | `roi_crop` | `cv_tools.filters.roi` | 1 |
| `roi_filter` | `roi_filter` | `cv_tools.filters.roi` | — |
| `roi_draw` | `roi_draw` | `cv_tools.filters.roi` | 1 |
| `crop` | `crop` | `cv_tools.filters.crop_resize` | 1 |
| `resize` | `resize` | `cv_tools.filters.crop_resize` | 1 |
| `rotate` | `rotate` | `cv_tools.filters.crop_resize` | 1 |
| `flip` | `flip` | `cv_tools.filters.crop_resize` | 1 |
| `sharpen` | `unsharp_mask` | `cv_tools.filters.sharpen` | 2 |
| `sharpen_laplacian` | `laplacian_sharpen` | `cv_tools.filters.sharpen` | 2 |
| `gaussian_blur` | `gaussian_blur` | `cv_tools.filters.smoothing` | 2 |
| `median_filter` | `median_filter` | `cv_tools.filters.smoothing` | 2 |
| `bilateral_filter` | `bilateral_filter` | `cv_tools.filters.smoothing` | 2 |

| | | | |
|---|---|---|---|
| `canny` | `canny_edges` | `cv_tools.filters.edge_detection` | 2 |
| `auto_canny` | `auto_canny` | `cv_tools.filters.edge_detection` | 2 |
| `sobel` | `sobel_edges` | `cv_tools.filters.edge_detection` | 2 |
| `laplacian` | `laplacian_edges` | `cv_tools.filters.edge_detection` | 2 |
| `ela` | `error_level_analysis` | `cv_tools.filters.ela` | 3 |
| `fft_spectrum` | `fft_magnitude_spectrum` | `cv_tools.filters.fft_analysis` | 3 |
| `fft_filter` | `fft_filter` | `cv_tools.filters.fft_analysis` | 3 |
| `remove_periodic` | `remove_periodic_noise` | `cv_tools.filters.fft_analysis` | 3 |
| `noise_map` | `noise_map` | `cv_tools.filters.noise_analysis` | 3 |
| `clone_detect` | `highlight_clones` | `cv_tools.filters.clone_detection` | 3 |
| `ghost` | `ghost_map` | `cv_tools.filters.jpeg_ghost` | 3 |
| `deblur_motion` | `deblur_motion` | `cv_tools.filters.motion_deblur` | 4 |
| `deblur_defocus` | `deblur_defocus` | `cv_tools.filters.motion_deblur` | 4 |
| `deblur_motion_grid` / `deblur_defocus_grid` | `deblur_motion_grid` / `deblur_defocus_grid` | `cv_tools.filters.motion_deblur` | — |
| `curves` / `s_curve` | `apply_curve` / `s_curve` | `cv_tools.filters.curves` | — |
| `white_balance` / `white_balance_patch` / `temperature` | `auto_white_balance` / `white_balance_from_patch` / `adjust_temperature` | `cv_tools.filters.white_balance` | — |
| `saturation` / `vibrance` / `desaturate` / `selective_saturation` | voir le module | `cv_tools.filters.saturation` | — |
| `color_balance` / `cmyk` / `channel_mixer` | voir le module | `cv_tools.filters.color_balance` | — |
| `invert` / `invert_channel` / `invert_luminance` / `solarize` | voir le module | `cv_tools.filters.invert` | — |
| `nl_means` / `nl_means_auto` | `nl_means_denoise` | `cv_tools.filters.nl_means_denoise` | — |

| | | |
|---|---|---|
| `upscale` | `upscale` | `cv_tools.filters.super_resolution` |
| `local_contrast` / `detail_enhance` / `multiscale_detail` / `texture_boost` | voir le module | `cv_tools.filters.detail_enhancement` |
| `perspective` / `auto_perspective` | `correct_perspective` | `cv_tools.filters.perspective_correction` |
| `barrel` / `fisheye` | `correct_barrel_distortion` / `correct_fisheye` | `cv_tools.filters.fisheye_correction` |
| `pixel_aspect` / `fit_aspect` | `correct_pixel_aspect` / `fit_to_aspect` | `cv_tools.filters.aspect_ratio` |
| `undistort` | `undistort_with_file` | `cv_tools.filters.undistort` |
| `clahe_grid` | `apply_clahe_grid` | `cv_tools.filters.clahe` — |
| `blocking_map` / `deblock` | voir le module | `cv_tools.filters.compression_analysis` |
| `stain` | `extract_stain` | `cv_tools.filters.color_deconvolution` |
| `component` / `bit_plane` | `extract_component` / `extract_bit_plane` | `cv_tools.filters.component_separation` |
| `redact` | `redact_region` | `cv_tools.filters.redaction` |

# Sprint 1 — Ajuster & Corriger

## CLAHE — `clahe`

Amélioration adaptative du contraste appliquée par tuiles, avec une limitation du contraste pour éviter d'amplifier le bruit. Le filtre de référence pour les images de vidéosurveillance sombres ou à faible contraste.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `clip_limit` | float | `2.0` | Plus élevé = plus de contraste local, plus de bruit |
| `tile_grid_size` | int ou (lignes, colonnes) | `8` | `8` signifie des tuiles de 8×8 |
| `color_mode` | str | `'lab'` | `lab`, `hsv`, `yuv`, `channelwise`, `luminance` |

`lab`, `hsv` et `yuv` égalisent un seul canal de luminance et laissent la chrominance inchangée, la couleur reste donc stable. `channelwise` égalise R, G et B indépendamment et **modifiera** la couleur — à n'utiliser que si c'est l'effet recherché.

CLI : `--clahe clip=3.0 tile=8x8 mode=lab` Sur la profondeur : une source 10 ou 12 bits arrive en `uint16` et est égalisée en 16 bits par `yuv`, `channelwise` et `luminance`. `lab` et `hsv` lèvent une erreur sur une entrée 16 bits, la conversion OpenCV correspondante n'acceptant que le 8 bits — convertissez l'image vous-même en acceptant la perte, ou choisissez un mode qui tient la profondeur.

`apply_clahe_grid(image, clip_limits, tile_grid_sizes)` génère une grille annotée de combinaisons de paramètres pour choisir des valeurs rapidement. Enregistré sous le nom `clahe_grid`, donc offert par les deux interfaces ; les deux arguments ont des valeurs par défaut utilisables et acceptent une valeur unique. Le coût en bruit d'un `clip_limit` donné variant d'un facteur 1,4 à 1,9 selon l'image, la valeur se choisit sur la planche plutôt qu'au jugé devant un curseur.

## Contraste & Luminosité — `contrast_brightness`

`sortie = (entrée - 128) * contraste + 128 + luminosité`, puis gamma.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `brightness` | float | `0.0` | Décalage, de -255 à 255 |
| `contrast` | float | `1.0` | 1.0 = inchangé |
| `gamma` | float | `1.0` | <1 assombrit les tons moyens, >1 les éclaircit |

| | | | |
|---|---|---|---|
| `channel` | str ou None | `None` | `r`, `g`, `b` pour cibler un seul canal |

CLI : `--brightness 30`, `--contrast 1.5`, `--gamma 0.8` (chacun est une étape distincte de la chaîne)

## Contraste automatique — `auto_contrast`

Étire l'histogramme de luminance sur toute la plage.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `cutoff` | float | `0.0` | Pourcentage des pixels les plus sombres/clairs à ignorer (0–50) |

CLI : `--auto-contrast` ou `--auto-contrast 2`

## Niveaux — `levels`

Fait correspondre la plage d'entrée `[black_point, white_point]` à la plage de sortie `[output_black, output_white]` avec une courbe gamma sur les tons moyens.

| Paramètre | Type | Défaut |
|---|---|---|
| `black_point` | float | `0` |
| `gamma` | float | `1.0` |
| `white_point` | float | `255` |
| `output_black` | float | `0` |
| `output_white` | float | `255` |
| `channel` | str ou None | `None` |

Lève une `ValueError` si `black_point >= white_point`.

CLI : `--levels 20,1.0,220` (noir, gamma, blanc)

## Niveaux automatiques — `auto_levels`

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `per_channel` | bool | `False` | `True` étire R/G/B indépendamment et peut altérer la couleur |

CLI : `--auto-levels`

## Égalisation d'histogramme — `histeq`

Égalisation globale — aplatit tout l'histogramme d'un coup. Plus forte et plus brutale que CLAHE ; a tendance à amplifier le bruit dans les zones plates.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `color_mode` | str | `'lab'` | `lab`, `hsv`, `yuv`, `channelwise`, `grayscale` |
| `mask` | ndarray ou None | `None` | Restreint la région utilisée pour construire l'histogramme |

CLI : `--histeq mode=lab`

## Filtre sur ROI — `roi_filter`

Applique un autre filtre enregistré à une seule région et laisse le reste de l'image intact, parce que le reste n'est pas ce qu'on démontre. C'est aussi la vraie réponse à l'histogramme bimodal : appliquée à la zone qui porte la question, une opération de contraste travaille sur l'histogramme qui intéresse.

| paramètre | type | défaut | notes |
|---|---|---|---|
| `x`, `y`, `width`, `height` | int | — | La région, rognée aux bornes de l'image |
| `filter_name` | str | `'clahe'` | Nom du filtre à appliquer dans la région |
| `feather` | int | `8` | Largeur de la rampe de fondu en pixels ; `0` pour un bord net |

La transition est adoucie plutôt que coupée : une couture nette autour d'une zone rehaussée est une question à l'audience. La rampe est bornée par la région, de sorte qu'une petite ROI en reçoit une proportionnée.

Le filtre intérieur tourne avec ses propres valeurs par défaut — nommer un filtre *et* ses paramètres demanderait des paramètres imbriqués, ce que le contrat JSON plat du registre n'autorise pas. Seuls les filtres qui tournent sur une image seule peuvent être imbriqués ; les autres sont refusés par leur nom plutôt qu'en échouant plus loin. Un filtre qui redimensionne la région est refusé également.

## Recadrage ROI — `roi_crop`

Recadre sur une région, **limitée** aux bords de l'image — une région surdimensionnée se réduit silencieusement.

| Paramètre | Type |
|---|---|
| `x`, `y`, `width`, `height` | int |

CLI : `--roi 100,100,300,200`

## Dessin ROI — `roi_draw`

Dessine un rectangle pour marquer une région sans altérer les pixels en dessous (sauf si `filled`).

| Paramètre | Type | Défaut |
|---|---|---|
| `x`, `y`, `width`, `height` | int | — |
| `color` | (r, g, b) | `(255, 0, 0)` |
| `thickness` | int | `2` |
| `label` | str ou None | `None` |
| `filled` | bool | `False` |
| `alpha` | float | `0.3` |

CLI : `--draw-roi 265,295,120,46`

## Recadrage — `crop`

Même géométrie que `roi_crop`, mais **lève** une `ValueError` lorsque la région tombe entièrement hors de l'image, au lieu de renvoyer un résultat tronqué. À utiliser quand un recadrage hors limites doit être une erreur.

CLI : `--crop 100,100,300,200`

## Redimensionnement — `resize`

| Paramètre | Type | Remarques |
|---|---|---|
| `width` | int ou None | Seul, la hauteur suit le ratio d'aspect |
| `height` | int ou None | Seule, la largeur suit le ratio d'aspect |
| `scale` | float ou None | Utilisé en l'absence de width/height |
| `interpolation` | str | `auto`, `nearest`, `bilinear`, `bicubic`, `lanczos`, `area` |

`auto` choisit `INTER_AREA` pour la réduction et `INTER_LANCZOS4` pour l'agrandissement. Pour un travail forensique, préférer `nearest` quand il faut inspecter les pixels sans rééchantillonnage.

CLI : `--resize 800x600`, `--resize 800x`, `--resize x600`, `--resize 50%`, `--resize 0.5`, combiné avec `--interpolation lanczos`.

## Rotation — `rotate`

Agrandit le canevas pour qu'aucun contenu ne soit tronqué.

| Paramètre | Type | Défaut |
|---|---|---|
| `angle` | float | — (degrés, sens antihoraire) |
| `center` | (x, y) ou None | `None` = centre de l'image |
| `scale` | float | `1.0` |
| `border_mode` | str | `'constant'` (`replicate`, `reflect`, `wrap`) |
| `border_value` | (r, g, b) | `(0, 0, 0)` |

CLI : `--rotate 90`

## Retournement — `flip`

| Paramètre | Type | Valeurs |
|---|---|---|
| `direction` | str | `horizontal`, `vertical`, `both` |

CLI : `--flip horizontal`

---

# Sprint 2 — Améliorer

## Masque flou (Unsharp Mask) — `sharpen`

Soustrait une copie floutée pour isoler le détail, puis le rajoute pondéré par `amount`.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `amount` | float | `1.0` | 0 = aucun changement ; au-delà de 2, l'effet paraît généralement artificiel |
| `radius` | float | `1.0` | Sigma du flou. Petit = détail fin, grand = contraste local. |
| `threshold` | int | `0` | Contraste local minimal (0–255) avant qu'un pixel ne soit accentué |

`threshold` est le contrôle du bruit : l'augmenter laisse tranquilles les zones lisses — là où vit le bruit — tout en accentuant les véritables contours. Accentuer **après** le débruitage, jamais avant.

CLI : `--sharpen amount=1.5 radius=1.0 threshold=4`

`sharpen_grid(image, amounts, radii)` génère une grille annotée de paramètres, comme `apply_clahe_grid`.

## Accentuation laplacienne — `sharpen_laplacian`

`sortie = entrée - force × laplacien(entrée)`. Plus dur et plus sensible au bruit qu'un masque flou, mais ne nécessite pas de choisir un rayon.

| Paramètre | Type | Défaut |
|---|---|---|
| `strength` | float | `1.0` |
| `kernel_size` | int | `3` (impair, 1–31) |

CLI : `--sharpen-laplacian strength=1.0 kernel=3`

## Flou gaussien — `gaussian_blur`

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `radius` | float | `2.0` | Sigma gaussien en pixels |

| | | | |
|---|---|---|---|
| `kernel_size` | int | `0` | 0 dérive le noyau à partir du rayon — c'est normalement ce qu'il faut |

Lissage généraliste. Floute les contours en même temps que le bruit ; préférer le bilatéral quand les contours importent.

CLI : `--gaussian 1.5` (`--gaussian` seul utilise 2.0)

## Filtre médian — `median_filter`

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `kernel_size` | int | `3` | Impair, 3 ou plus |

Le remède standard contre le bruit impulsionnel (poivre et sel). La sortie ne contient jamais que des valeurs déjà présentes à proximité, donc les plateaux et les contours survivent intacts là où un flou les aurait étalés.

CLI : `--median 3` (`--median` seul utilise 3)

## Filtre bilatéral — `bilateral_filter`

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `diameter` | int | `9` | Diamètre du voisinage ; 0 le dérive de `sigma_space` |
| `sigma_color` | float | `75.0` | Tolérance à la différence de couleur |
| `sigma_space` | float | `75.0` | Étendue spatiale |

Pondère les voisins à la fois par la distance et par la similarité de couleur, ce qui permet de lisser le bruit du capteur à l'intérieur des régions sans déborder sur les frontières. Le plus lent des trois filtres de lissage.

CLI : `--bilateral d=9 color=75 space=75`

---

# Sprint 2 — Analyser

Les quatre détecteurs de contours renvoient tous une carte **mono-canal** en uint8, donc ils convertissent une image couleur en niveaux de gris en cours de chaîne.

## Canny — `canny`

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `low_threshold` | float | `100` | Les contours faibles ne survivent que s'ils sont connectés à un contour fort |
| `high_threshold` | float | `200` | Un ratio bas:haut de 1:2 ou 1:3 est le point de départ habituel |
| `aperture_size` | int | `3` | 3, 5, ou 7 |
| `l2_gradient` | bool | `False` | Magnitude L2 exacte plutôt que l'approximation L1, moins coûteuse |
| `blur_sigma` | float | `0.0` | Flou gaussien préalable — nécessaire sur des images bruitées |

La sortie est binaire (0 ou 255).

CLI : `--canny 50,150`, avec `--blur-first 1.5` pour régler le flou préalable.

## Auto Canny — `auto_canny`

Dérive les deux seuils à partir de l'intensité médiane de l'image, ce qui convient à des lots d'images dont l'exposition varie. Revient à 50/150 lorsque la médiane fait coïncider les deux seuils (une image quasi noire ou quasi blanche).

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `sigma` | float | `0.33` | Écart autour de la médiane, 0–1. Plus grand conserve plus de contours. |
| `blur_sigma` | float | `0.0` | Flou gaussien préalable |

CLI : `--auto-canny` ou `--auto-canny 0.4`

## Sobel — `sobel`

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `dx` | int | `1` | Ordre de la dérivée horizontale (0 ou 1) |
| `dy` | int | `1` | Ordre de la dérivée verticale (0 ou 1) |
| `kernel_size` | int | `3` | Impair, 1–7 |
| `normalize` | bool | `True` | Étire le résultat pour remplir 0–255 |

Avec `dx` et `dy` réglés tous les deux, on obtient la magnitude du gradient ; avec un seul, on obtient cette dérivée directionnelle unique. Valeurs continues, contrairement à la sortie binaire de Canny.

CLI : `--sobel dx=1 dy=1 kernel=3`

## Laplacien — `laplacian`

| Paramètre | Type | Défaut |
|---|---|---|
| `kernel_size` | int | `3` (impair, 1–31) |
| `normalize` | bool | `True` |
| `blur_sigma` | float | `0.0` |

Répond au changement d'intensité dans toutes les directions à la fois — et au bruit tout aussi volontiers, donc un léger `blur_sigma` vaut généralement le coup.

CLI : `--laplacian kernel=3 blur=1.0`

## Histogramme (pas une étape de chaîne)

`histogram.py` analyse plutôt qu'il ne transforme, il est donc exposé sur la CLI comme des options de sortie plutôt que comme des filtres du registre.

| Fonction | Renvoie |
|---|---|
| `compute_histogram(image, bins=256, normalize=False)` | Dictionnaire nom de canal → comptes par bin |
| `histogram_stats(image)` | Moyenne, médiane, écart-type, min, max, p1, p99, % d'écrêtage par canal |

| | |
|---|---|
| `dynamic_range_used(image)` | Fraction de 0–255 couverte entre p1 et p99 |
| `render_histogram(image, ...)` | Image du graphique en RVB |

Les pourcentages d'écrêtage sont la partie forensiquement importante : les pixels bloqués à 0 ou 255 ont perdu leurs valeurs d'origine, et aucune amélioration ne les récupère. Un faible `dynamic_range_used` signifie que les niveaux ou le CLAHE ont encore de la marge.

`render_histogram` accepte `width`, `height`, `bins`, `log_scale` (révèle les queues éparses qu'un tracé linéaire aplatit à néant), `show_grid`, `background`, et `channels` pour restreindre les courbes tracées.

CLI : `--histogram chart.png`, `--histogram-log`, `--hist-stats`

`edge_density(edges, threshold=0)` donne la fraction de pixels porteurs d'un contour. Elle compare la netteté entre images d'une même scène, mais ce n'est pas une mesure générale de netteté — flouter une image dont la seule caractéristique est un contour marqué étale ce contour sur plus de pixels et augmente la densité.

---

# Sprint 3 — Forensique

**À lire avant d'utiliser l'un de ces filtres.** Ils repèrent des éléments *qui méritent d'être examinés*. Aucun d'eux n'établit qu'une image a été manipulée, et chacun a un mode d'échec qui produit une preuve d'apparence convaincante pour une conclusion fausse. Les mises en garde ci-dessous font partie intégrante de l'outil, ce ne sont pas de simples avertissements autour de lui.

## Analyse du niveau d'erreur (ELA) — `ela`

Recompresse l'image en JPEG et amplifie la différence. Une région dont l'historique de compression diffère de son entourage peut présenter un niveau d'erreur différent.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `quality` | int | `90` | Qualité de recompression, 1–100. Viser une valeur proche de l'originale. |
| `scale` | float | `0` | Multiplicateur de luminosité ; 0 met à l'échelle automatiquement pour que le pic d'erreur atteigne 255 |
| `grayscale` | bool | `False` | Regroupe l'erreur par canal en un seul canal |

**Limites.** Significatif uniquement sur un original JPEG — un réenregistrement en PNG, ou un second enregistrement JPEG complet de l'image, efface totalement le signal. Les zones claires suivent la densité de contours et de texture autant que l'historique d'édition, donc les régions chargées paraissent toujours « chaudes ». Une carte propre ne signifie pas que l'image est authentique.

CLI : `--ela quality=90 gray=true`, `--ela-stats [QUALITY]`

`ela_stats(image, quality, block_size)` renvoie l'erreur moyenne/maximale, une grille de moyennes par bloc, et le bloc le plus chaud avec un score z indiquant à quel point il se distingue du reste. `recompress(image, quality)` expose seul l'aller-retour JPEG.

## Spectre de magnitude FFT — `fft_spectrum`

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `log_scale` | bool | `True` | Sans cela, le terme DC écrase tout et le tracé n'est qu'un point |
| `normalize` | bool | `True` | Étire pour remplir 0–255 |

Le DC est au centre. Les structures périodiques — entrelacement, trames de similigravure, bandes de scanner — apparaissent sous forme de points lumineux discrets éloignés de ce centre.

CLI : `--fft log=true`

## Filtre fréquentiel — `fft_filter`

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `filter_type` | str | `'lowpass'` | `lowpass`, `highpass`, `bandpass` |
| `cutoff` | float | `30.0` | Rayon en pixels depuis le DC |
| `cutoff_high` | float | `0.0` | Rayon supérieur, bandpass uniquement |
| `soft` | bool | `True` | Masque à bord gaussien. Un bord dur provoque un effet de sonnerie qui ressemble à un vrai contenu d'image. |

Renvoie une sortie mono-canal.

CLI : `--fft-filter type=highpass cutoff=20`

## Suppression de bruit périodique — `remove_periodic`

Trouve les pics spectraux isolés et les élimine par filtre coupe-bande, supprimant un motif répétitif avec bien moins de dégâts qu'un flou. Le pic miroir est également éliminé, car le spectre d'une image réelle est symétrique par rapport au DC.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `peaks` | list ou None | `None` | Détectés automatiquement si omis |
| `notch_radius` | float | `4.0` | Rayon du coupe-bande gaussien sur chaque pic |
| `min_radius` | float | `10.0` | Ignore ce rayon autour du DC — les basses fréquences sont l'image elle-même |
| `threshold` | float | `4.0` | Écarts-types au-dessus du fond local pour qu'un pic compte |

Un résidu du motif survit généralement sur les bords de l'image : la FFT traite l'image comme si elle pavait le plan, et la discontinuité aux bords disperse de l'énergie dans tout le spectre.

CLI : `--remove-periodic notch=4`

`detect_periodic_peaks(image, ...)` renvoie les pics seuls, pour les inspecter avant l'élimination.

## Carte de bruit — `noise_map`

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `block_size` | int | `32` | Plus petit localise mieux mais estime chaque bloc moins fiablement |
| `normalize` | bool | `True` | Étire pour remplir 0–255 |
| `upscale` | bool | `True` | Redimensionne la grille de blocs aux dimensions de l'image d'entrée |

Plus clair signifie plus bruité. Le bruit du capteur devrait être assez uniforme sur une image non retouchée, donc une région qui diffère nettement provient d'ailleurs — un capteur différent, un redimensionnement différent, ou un débruitage appliqué à cette seule région. Une forte texture augmente aussi la mesure.

CLI : `--noise-map 32`, `--noise-stats`

`estimate_noise(image)` renvoie le sigma global via la méthode d'Immerkaer ; `estimate_snr` donne un résultat en dB (en utilisant l'écart-type de l'image elle-même comme signal, donc une image plate obtient un score bas même si elle est propre) ; `noise_report` ajoute des statistiques par bloc et un ratio d'`uniformity`.

## Détection de clonage — `clone_detect`

Trouve les régions dupliquées d'ailleurs dans la même image. Les blocs sont décrits par leurs coefficients DCT de basse fréquence, triés de sorte que les blocs quasi identiques deviennent voisins, et un décalage partagé par de nombreuses paires est signalé comme une région dupliquée.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `step` | int | `1` | **Seuls les décalages multiples de cette valeur sont détectables** |
| `block_size` | int | `16` | Côté de chaque bloc comparé |
| `coefficients` | int | `4` | Taille du carré supérieur gauche de coefficients DCT conservé comme descripteur |

| | | | |
|---|---|---|---|
| `quantization` | float | `4.0` | Arrondi du descripteur ; plus grand tolère plus de compression, mais produit plus de fausses correspondances |
| `min_distance` | float | `0` | Séparation minimale pour qu'une paire compte ; 0 utilise 2×`block_size` |
| `min_matches` | int | `8` | Nombre de paires requis avant qu'un décalage soit signalé |
| `min_variance` | float | `12.0` | Les blocs sans relief en dessous de ce seuil sont ignorés |
| `max_blocks` | int | `300000` | Garde-fou mémoire ; lève une erreur plutôt que d'allouer des gigaoctets |

**La contrainte `step` est celle qui piège le plus souvent.** Les blocs sont échantillonnés sur une grille de ce pas, donc une région déplacée de 190 pixels est invisible pour un pas de 8 — la copie est échantillonnée à une phase différente de l'original et les descripteurs ne correspondent pas. La valeur par défaut de 1 est exhaustive. L'augmenter donne un passage de dépistage rapide, pas une recherche complète.

Sur une grande image, recadrer sur une région avec `--roi` plutôt que d'augmenter `step`.

**Limites.** Une répétition authentique — mur de briques, fenêtres, carrelage, texte — est de la duplication, et sera signalée. Cet outil localise la duplication, pas l'intention.

CLI : `--clone-detect block=16 step=1 matches=8 variance=12`, `--clone-stats`

`detect_copy_move` renvoie le dictionnaire de résultat complet (masque, vecteurs de décalage, comptes de blocs) ; `draw_clone_regions` le superpose en teinte sur l'image.

## Détection de fantôme JPEG — `ghost`

Recompresse l'image sur une plage de qualités JPEG et calcule la différence de chaque passage par rapport à la source, la même astuce que l'ELA utilise une seule fois. Requantifier une région déjà en JPEG à sa qualité antérieure est quasi sans perte, donc l'erreur d'une région chute à la qualité à laquelle elle a été enregistrée pour la dernière fois — le « fantôme ».

**Il faut indiquer la région.** Cet outil ne la cherche pas, et la raison est mesurée plutôt que supposée. Sur 18 images de vidéosurveillance intactes, la région la plus contrastée qu'une recherche automatique puisse trouver obtient –0,53 en moyenne, tandis que les mêmes images portant un collage Q55 réel obtiennent –0,44 : les images intactes se séparent *plus nettement* que les images truquées, parce qu'un mur plat forme, dans chaque image, une vaste région cohérente à faible différence à l'une ou l'autre qualité. Une recherche renvoie de la texture, pas une histoire. Une fois la région donnée, la même mesure est décisive : –0,34 à la qualité réelle contre –0,02 pour un témoin intact.

Marquez la région — la visionneuse de bureau remplit `x, y, width, height` à partir d'un glissé — et l'outil indique à quelle qualité elle a été compressée.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `qualities` | list[int] | `50,55,...,100` | Paliers de qualité croissants à balayer |
| `block_size` | int | `16` | Côté des blocs d'analyse |
| `region` | (x,y,w,h) | `None` | La région à interroger. Sans elle, rien n'est affirmé |
| `upscale` | bool | `True` | `ghost_map` seulement : redimensionne la grille de blocs à la taille de l'image |

**Précision mesurée.** Sur 12 images réelles portant un collage Q55 connu : détecté dans 10, et 2 images intactes déclarées positives. Une détection est une piste à examiner, jamais une conclusion. La qualité retrouvée tombe à un palier près de la vérité : lisez-la comme un voisinage, pas comme un nombre.

**Limites.** Un nouvel enregistrement JPEG uniforme de tout le montage est un angle mort : chaque bloc partage alors une même qualité finale réelle, et son creux quasi nul à cette qualité noie toute trace plus subtile de la compression antérieure d'une région avant un collage. La technique lit un montage qui n'a jamais été unifié par un enregistrement JPEG ultérieur sur l'image entière — un PNG construit à partir de sources JPEG est le cas courant qu'elle détecte. Les régions plates et peu texturées ne creusent que faiblement à chaque qualité et se lisent comme ambiguës par construction. Le seuil de 0,10 a été calibré sur une seule caméra ; recalibrez-le avant de vous y fier ailleurs.

CLI : `--ghost block=16 min=50 max=100 step=5`, `--ghost-stats`

`ghost_sweep` renvoie la différence normalisée à chaque qualité — la matière première, et la forme qu'il vaut la peine de regarder. `ghost_map` renvoie l'unique image du balayage qui porte le plus de structure, sombre là où les pixels correspondent à cette qualité. `ghost_report` nomme la qualité de la région, l'écart qui a tranché, et le score de chaque qualité pour que le verdict puisse être vérifié plutôt que cru.

> Une version antérieure de ce filtre prenait le minimum *global* de chaque bloc sur le balayage. Cela ne peut pas fonctionner : la courbe de différence décroît de façon monotone vers la qualité 100 pour presque tous les blocs, si bien que le minimum tombe en haut du balayage quelle que soit l'histoire du bloc, et une image intacte déclarait 42 % de ses blocs atypiques. Les préréglages et rapports écrits avant ce correctif mentionnent `dominant_quality` et `outlier_count`, qui n'existent plus.
>
> **Sur les filtres qui ne trouvent rien.** `clone_detect` et `auto_perspective` renvoient l'image inchangée lorsqu'ils ne détectent aucune région dupliquée ni aucun quadrilatère. C'est le bon contrat pour une étape de chaîne — une image entre, une image sort, et un préréglage se rejoue à l'identique — mais « rien trouvé » et « rien fait » se ressemblent dans la visionneuse. Utilisez le rapport correspondant pour les distinguer : `--clone-stats` indique explicitement *no duplicated regions found*, et l'onglet Analysis de l'interface graphique affiche la même ligne.

## Analyse des métadonnées (pas une étape de chaîne)

Lit le conteneur plutôt que les pixels : les tags EXIF, les segments applicatifs JPEG, et la concordance entre ce que les métadonnées annoncent et ce que l'image est réellement. `metadata_report(path)` prend un chemin de fichier, pas une image, et constitue donc une option de statistiques plutôt qu'un filtre.

| Contrôle | Gravité | Signification |
|---|---|---|

| | | |
|---|---|---|
| `editing_software` | flag | `Software` nomme un éditeur connu (comparé à `EDITOR_SIGNATURES`, afin que les chaînes de firmware d'appareil photo ne le déclenchent pas) |
| `modified_after_capture` | flag | `DateTime` est postérieur à `DateTimeOriginal` — le fichier a été réécrit après le déclenchement |
| `timestamp_disorder` | flag | `DateTimeDigitized` précède `DateTimeOriginal`, ce que l'ordre de capture interdit |
| `dimension_mismatch` | flag | Les dimensions enregistrées dans l'EXIF diffèrent des dimensions réelles — image redimensionnée ou recadrée depuis la capture |
| `photoshop_segment` | flag | Un bloc de ressources Photoshop APP13 est intégré |
| `thumbnail_mismatch` | flag | Le contenu de la miniature EXIF intégrée diverge de l'image principale |
| `no_exif` | info | Un format qui porte normalement de l'EXIF n'en a aucun |
| `no_camera_identification` | info | EXIF présent mais sans `Make` ni `Model` |
| `xmp_segment` | info | Un paquet XMP est intégré ; il consigne souvent un historique d'édition absent de l'EXIF |

**C'est le contrôle le moins coûteux qui soit, et le plus facile à déjouer.** Les métadonnées sont du texte brut dans un en-tête : n'importe qui peut les modifier ou les supprimer, et la plupart des messageries et réseaux sociaux les suppriment intégralement au téléversement. Un en-tête propre ne prouve donc rien — c'est l'état normal d'un fichier passé par WhatsApp — et le nom d'un éditeur ne prouve rien non plus, puisque recadrer, pivoter ou convertir en laissent tous un.

Ce sont les contradictions qui méritent l'attention. Un tag qui contredit les pixels, ou un autre tag, est plus difficile à produire par accident qu'un nom d'apparence suspecte.

**La miniature intégrée est une contradiction que les éditeurs laissent souvent derrière eux.** Les JPEG transportent une seconde copie réduite de l'image dans l'IFD1 de l'EXIF, destinée aux aperçus, et un éditeur qui remplace les pixels n'a aucune raison de la régénérer — un recadrage, un montage ou un remplacement du sujet peut laisser la miniature afficher encore la scène d'origine. `check_thumbnail_mismatch` l'extrait et compare son contenu à l'image principale au moyen d'un hachage perceptuel simple (un average-hash 8x8), tolérant à la recompression propre de la miniature mais pas à une image réellement différente. L'absence de miniature n'est pas en soi un signalement — c'est l'état normal de nombreux fichiers ordinaires.

CLI : `--metadata-stats`

`read_exif` renvoie les tags sous forme de dictionnaire ; `detect_editing_software` et `check_timestamps` sont les contrôles individuels, utilisables sur un dictionnaire EXIF déjà en main. `extract_thumbnail` renvoie les octets JPEG bruts de la miniature intégrée, ou `None`.

## Défloutage — `deblur_motion`, `deblur_defocus`

Inverse un flou connu. L'inversion naïve divise par la réponse fréquentielle du flou, qui est proche de zéro à certaines fréquences, ce qui amplifie le bruit à ces fréquences sans limite. Le filtre de Wiener tempère cela : `F = G · conj(H) / (|H|² + K)`, où `K` est `noise_power`.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `length` | float | `15.0` | Étendue du mouvement en pixels : la distance entre les deux images fantômes d'un même point. Les fractions comptent (`deblur_motion`) |
| `angle` | float | `0.0` | Direction du mouvement en degrés, 0 = horizontal, sens antihoraire (`deblur_motion`) |
| `thickness` | float | `1.0` | Largeur de la traînée en pixels ; à augmenter quand l'image est aussi légèrement floue (`deblur_motion`) |
| `radius` | float | `5.0` | Rayon du cercle de défocalisation, à partir de 0,5. Les fractions comptent (`deblur_defocus`) |
| `noise_power` | float | `0.01` | Augmenter sur des images bruitées pour échanger de la netteté contre de la stabilité. Même sens pour les deux solveurs |
| `solver` | str | `wiener` | `wiener` ou `richardson_lucy` ; voir *Solveur* ci-dessous |
| `iterations` | int | `40` | Nombre de passes de correction, `richardson_lucy` uniquement |

Les deux PSF sont échantillonnées en surface : un pixel que la ligne ou le cercle ne couvre qu'en partie porte cette fraction de son poids, si bien que `length=15.4` et `angle=17` ne sont pas arrondis au pixel entier le plus proche. Ce n'est pas cosmétique : une PSF trop longue de deux pixels, ou décalée de cinq degrés, restaure moins bien que de laisser l'image floue.

La FFT traite l'image comme un motif qui pave le plan : son bord droit rejoint son bord gauche. Là où ils diffèrent, c'est une marche que la scène n'a jamais contenue, et la déconvolution produit des oscillations qui s'étendent à toute l'image. L'entrée est donc prolongée par une marge dans laquelle chaque bord se fond dans le bord opposé, puis recadrée. La bande située à environ une longueur de flou du bord reste la partie la moins fiable du résultat : ce qui se trouvait hors champ y a été mélangé par le flou, et rien ne permet de le retrouver.

La luminosité moyenne de l'image est conservée quelle que soit la valeur de `noise_power`.

## Solveur

`wiener` est le solveur par défaut, celui que décrit la formule ci-dessus : un seul filtre linéaire, appliqué une fois. Chaque pixel de sortie est une somme pondérée fixe de pixels d'entrée, les poids ne dépendent que de la PSF et de `noise_power`, et c'est une phrase que l'on peut écrire dans un rapport.

`richardson_lucy` est l'alternative pour les images bruitées et compressées, c'est-à-dire la plupart des images de vidéosurveillance. Il part d'une image uniforme et la corrige de façon répétée : flouter l'estimation, la comparer à l'image enregistrée, rapprocher l'estimation de celle-ci. Il fait mieux dans ce cas pour trois raisons.

- **L'estimation ne peut pas devenir négative.** L'essentiel des oscillations d'un résultat de Wiener vient du filtre qui descend sous le noir à côté d'un élément lumineux — un phare, une fenêtre éclairée. Cela ne peut pas se produire ici.
- **Il travaille en lumière linéaire**, là où un flou s'additionne réellement. Les valeurs enregistrées sont codées en gamma ; elles sont décodées en sRGB, déconvoluées puis recodées. Une caméra dont la courbe de tons s'éloigne du sRGB profite moins de cet avantage, sans que le résultat soit faux pour autant.
- **Ce qui se trouvait hors champ est estimé**, et non supposé ; la bande de bord est donc meilleure.

Un terme de variation totale (Dey et al., 2006), proportionnel à `noise_power`, contient le bruit à l'échelle du pixel que Richardson–Lucy accumule au fil des itérations.

Mesuré sur des captures de vidéosurveillance simulées — des images nettes floutées à une résolution inférieure au pixel en lumière linéaire, auxquelles sont ajoutés du bruit de capteur et une compression JPEG, de sorte que l'image réelle est connue. Gain moyen de PSNR par rapport à l'image floue, douze cas par colonne, pire cas entre parenthèses :

| propre | bruit + JPEG 70 | bruit + JPEG 45 | bruit + JPEG 25 |
|---|---|---|---|
| `wiener` `noise_power` 0,01 | 2,9 dB (2,0) | 1,8 (0,2) | 1,3 (–0,6) | 0,6 (–1,6) |
| `wiener` `noise_power` 0,03 | 2,2 (1,3) | 1,7 (0,8) | 1,5 (0,4) | 1,2 (–0,1) |

| | | | | |
|---|---|---|---|---|
| `richardson_lucy`, valeurs par défaut | 3,8 (2,3) | 3,3 (2,1) | 2,9 (1,5) | 2,4 (0,8) |

Il tolère aussi mieux une PSF légèrement fausse : avec une longueur erronée de 1,5 pixel ou un angle erroné de 4 degrés sur les captures JPEG 45, il conservait 2,8 à 3,1 dB de gain là où Wiener en conservait 1,2 à 1,6.

**Tout cela dépend de l'endroit où le flou s'est produit.** Un objectif ou un sujet en mouvement floute la lumière sur le capteur, avant que la caméra ne la code en gamma, et les chiffres ci-dessus valent pour un flou produit de cette façon. Un flou appliqué aux valeurs enregistrées — par un logiciel de retouche, ou par un test qui floute un JPEG — est une autre opération, et c'est celle que `wiener` suppose. Sur un flou produit ainsi, l'ordre s'inverse pour des images propres : `wiener` gagnait 3,1 dB et `richardson_lucy` 2,0, et à JPEG 60 les deux étaient à moins de 0,2 dB l'un de l'autre. Avec du bruit ajouté ou à JPEG 40, `richardson_lucy` repassait devant. Un flou que vous avez fabriqué vous-même pour essayer le filtre favorisera donc `wiener` ; des images réelles ne le devraient pas. Aucune séquence dont l'original soit connu n'était disponible pour le confirmer sur une vraie caméra, dont la courbe de tons n'est pas exactement le sRGB ; les deux cas ci-dessus sont les limites entre lesquelles elle se situe.

**Ce que cela coûte.** Le résultat n'est plus une somme pondérée fixe de l'entrée. Il est entièrement déterminé par l'image et les paramètres enregistrés et se reproduit à l'identique, mais c'est le produit d'une itération, et un expert doit être prêt à le dire. Il est aussi des dizaines de fois plus lent — environ 1,6 s sur une image de 640×362 contre 0,03 s, environ 11 s en 1920×1080 — donc recadrez d'abord. Les planches de comparaison acceptent aussi `solver`, mais la PSF qui convient à un solveur convient à l'autre : cherchez avec `wiener`, puis changez.

Les hautes lumières écrêtées ne demandent aucun réglage. Un traitement particulier des pixels écrêtés a été essayé — les ignorer, ou les considérer comme « au moins aussi lumineux » — et aucun des deux n'a donné de meilleure mesure que le solveur seul ; les ignorer creusait les grandes zones surexposées. Aucun n'a été conservé.

Le traitement en lumière linéaire a aussi été essayé pour `wiener`. Il gagnait 0,8 dB sur les captures propres, peu ou rien sur les captures compressées, et aggravait les pires cas ; `wiener` travaille donc sur les valeurs enregistrées, comme il l'a toujours fait.

Amped FIVE ne publie pas l'algorithme de ses filtres de défloutage. Les deux solveurs sont ici le choix, propre à cette boîte à outils, de méthodes classiques.

**Limites.** Il faut fournir la PSF correcte ; une longueur ou un angle devinés produisent un détail d'apparence confiante qui n'a jamais été enregistré. La déconvolution suppose aussi un flou uniforme sur toute l'image, donc une scène où un seul objet a bougé nécessite d'isoler d'abord cet objet avec `--roi`.

## Trouver la PSF

L'ordre de travail suit celui des filtres Motion Deblurring et Optical Deblurring d'Amped FIVE : recadrer sur le sujet, mesurer le flou, déflouter, et seulement ensuite ajuster les niveaux, accentuer ou corriger la perspective.

1. **Recadrer** sur la plaque, le visage ou le panneau. La déconvolution suppose un seul flou sur toute l'image, et les planches ci-dessous n'affichent leurs vignettes en taille réelle que tant que la planche tient dans la limite.
2. **Mesurer un flou de mouvement en traçant sa ligne.** Trouvez un point de la scène que le flou a dessiné deux fois — les deux images fantômes d'un coin de plaque, les deux extrémités de la traînée laissée par un point lumineux — et reliez-les. Dans l'une ou l'autre interface, sélectionnez `deblur_motion`, appuyez sur *Pick points on the image* et cliquez sur les deux fantômes : `length` et `angle` sont remplis. En ligne de commande, `--deblur line=X1,Y1,X2,Y2` fait la même chose. La ligne elle-même n'est pas conservée ; le préréglage et le rapport enregistrent la longueur et l'angle obtenus, c'est-à-dire la PSF réellement utilisée.
3. **Vérifier la fonction d'étalement du point.** Les deux interfaces dessinent la PSF à côté du formulaire pour chaque filtre de défloutage. Elle doit ressembler à la traînée visible dans l'image. `psf_preview(psf)` renvoie la même image.

4. **Affiner avec une planche de comparaison.** Une ligne tracée à la main est juste à un ou deux pixels et quelques degrés près, ce qui ne suffit pas. `deblur_motion_grid` présente l'estimation à côté de ses voisines — un pas plus court et plus long selon les lignes, un pas d'angle de part et d'autre selon les colonnes. Prenez les valeurs de la meilleure vignette, divisez les pas par deux et recommencez. La défocalisation ne laisse aucune ligne à tracer ; c'est donc avec `deblur_defocus_grid` que l'on trouve son rayon : le rayon selon les lignes, et le terme de bruit à un tiers, une fois et trois fois sa valeur selon les colonnes, car les deux se compensent.
5. **Déflouter**, puis ajuster.

| Paramètre | Type | Défaut | Remarques |
|---|---|---|---|
| `length_step` | float | `2.0` | Pixels entre les lignes (`deblur_motion_grid`) |
| `angle_step` | float | `5.0` | Degrés entre les colonnes (`deblur_motion_grid`) |
| `radius_step` | float | `1.0` | Pixels entre les lignes (`deblur_defocus_grid`) |

Les planches acceptent aussi les paramètres du filtre qu'elles comparent. Les vignettes sont les résultats eux-mêmes, non des miniatures, tant que la planche ne dépasse pas 2400 pixels de côté : les oscillations et le bruit amplifié sont ce qui distingue la bonne PSF d'une PSF presque bonne, et réduire une vignette les fait disparaître. Chaque étiquette est placée dans une bande au-dessus de sa vignette, et non par-dessus.

**Ordre.** Les défauts se corrigent dans l'ordre inverse de celui où la caméra les a introduits ; deux ordres font donc l'objet d'un avertissement — en ligne de commande, sous la chaîne dans le tableau de bord, et dans le panneau Info de l'interface de bureau. La chaîne s'exécute quand même.

- *Un défloutage de défocalisation après `perspective`, `auto_perspective`, `barrel`, `fisheye` ou `undistort`.* La défocalisation se produit dans l'objectif, sur l'image telle qu'elle est enregistrée, où le flou est un même cercle partout. Une déformation étire chaque partie de l'image différemment et plus aucun rayon unique ne convient ensuite. Le flou de mouvement est volontairement exclu : le sujet se déplace dans la scène, et sa traînée peut être plus rectiligne après redressement qu'avant.
- *Tout défloutage après un filtre d'accentuation.* L'accentuation modifie les contours d'une manière que la PSF ne décrit pas, et la déconvolution transforme cet écart en oscillations.

Tracer la ligne et prévisualiser la PSF reprennent la méthode d'Amped FIVE. Les planches de comparaison et l'avertissement sur l'ordre sont propres à cette boîte à outils.

CLI : `--deblur length=15.5 angle=30 thickness=1 noise=0.01`, `--deblur line=412,300,455,268`, `--deblur-defocus radius=5 noise=0.01`, `--deblur line=412,300,455,268 solver=richardson_lucy iterations=40`, `--deblur-grid line=412,300,455,268 length_step=1 angle_step=3`, `--deblur-defocus-grid radius=3 radius_step=0.5`

`motion_blur_psf` et `defocus_psf` construisent les PSF ; `apply_psf` en applique une vers l'avant, utile pour prévisualiser ce que signifie une PSF. `wiener_deconvolution` et `richardson_lucy_deconvolution` acceptent une PSF arbitraire ; `deconvolve` choisit entre eux par leur nom.

`deblur_sweep(image, lengths, angles)` est la planche de comparaison pour des longueurs et des angles quelconques, quand les candidats ne sont pas à un pas de part et d'autre d'une même estimation. `motion_from_line(start, end)` est la mesure que font les deux clics. `focus_score(image)` est la variance du laplacien ; elle augmente avec les oscillations autant qu'avec le détail retrouvé, de sorte qu'une mauvaise PSF peut obtenir un meilleur score que la bonne. Ne choisissez pas une PSF d'après elle.

# Sprint 3 — Multi-images (pas des étapes de chaîne)

`frame_averaging.py` consomme une *séquence* d'images et produit une seule image, il s'exécute donc avant la chaîne de filtres plutôt qu'à l'intérieur. Sur la CLI, c'est `--frames N`, avec `--frame` pour sélectionner l'index de départ et `--frame-step` pour le pas.

| Fonction | Méthode CLI | Ce qu'elle fait |
|---|---|---|
| `average_frames(frames, weights=None)` | `mean` | Supprime le bruit aléatoire ; le bruit diminue avec la racine carrée du nombre d'images |
| `median_frames(frames)` | `median` | Supprime tout ce qui est présent dans moins de la moitié des images, reconstruisant l'arrière-plan |
| `integrate_frames(frames, gain, auto_scale)` | `integrate` | Accumule la lumière d'images très sombres sans amplifier le bruit comme le ferait un gain |
| `sharpest_frames(frames, count)` | `sharpest` | Classe les images par netteté ; la CLI moyenne la meilleure moitié |

Toutes supposent que les images sont **alignées**, et aucune ne peut détecter qu'elles ne le sont pas — une caméra en mouvement renvoie simplement une image plus molle, sans rien dire de la raison.

`frame_difference(a, b, amplify)` donne la différence absolue entre deux images, pour isoler ce qui a bougé.

CLI : `--frames 24 --frame-method median --frame-step 5`

## Stabilisation — `--stabilise`

`stabilise.py` amène une séquence à un alignement commun, ce qui rend les méthodes ci-dessus utilisables sur des images qui n'ont pas été tournées sur trépied. Mesuré sur une séquence tremblante et bruitée de 16 images, comparée à l'original propre :

| | PSNR |
|---|---|
| une image bruitée | 17,86 dB |
| moyenne, sans alignement | 22,01 dB |
| **moyenne, après alignement** | **28,27 dB** |

Le moyennage seul récupère 4 dB de bruit puis plafonne, car il lutte désormais contre le mouvement de la caméra. L'alignement préalable en récupère 6 de plus — la différence entre une plaque illisible et une plaque lisible.

**Modèles de mouvement.** Choisir le moins de liberté que la caméra a réellement eu ; un modèle plus libre ajustera volontiers le bruit.

| Modèle | Pour |
|---|---|
| `translation` | Caméra fixe sur un support qui vibre |
| `euclidean` | Caméra à la main — tremblement et roulis (défaut) |
| `affine` | À la main, avec un léger changement d'échelle |
| `homography` | Panoramique ou zoom sur une scène globalement plane |

**Méthodes.** `features` apparie des points ORB et supporte les grands mouvements mais exige de la texture ; `ecc` maximise la corrélation, est précis au sous-pixel, et constitue une recherche *locale* qui ne converge que pour de petits mouvements sans point de départ. `auto` initialise `ecc` à partir de l'estimation par points et obtient donc les deux, en se repliant sur celle des deux moitiés qui a réussi.

Ce n'est pas le même travail que `super_resolution.estimate_shifts`, et les deux ne sont pas interchangeables : `estimate_shifts` mesure une translation pure à une petite fraction de pixel, ce dont une reconstruction a besoin avec une caméra quasi immobile. `align_frames` gère en plus la rotation, l'échelle et la perspective, au pixel près, ce qui rend combinables les images d'une caméra en mouvement. Stabiliser d'abord, reconstruire ensuite.

> **Un mauvais alignement étale, et le dissimule.** Une image qui n'a pas pu être appariée est pire qu'une image écartée, car le moyennage la cache — le résultat paraît simplement plus mou. Chaque image revient donc avec une confiance, celles sous `--stabilise-min-confidence` sont écartées plutôt que déformées au jugé, et `--report` consigne lesquelles. Attention : la confiance n'est pas une échelle unique — pour `ecc` c'est le coefficient de corrélation, pour `features` le taux d'inliers RANSAC. Les deux vont de 0 à 1 et plus c'est mieux, mais un 0,7 de l'un n'est pas un 0,7 de l'autre.

La sortie est **recadrée à la région couverte par toutes les images**. L'alignement décale et tourne chaque image, dont le bord opposé laisse une bande sans données ; moyenner sur cette bande mêle de vrais pixels à du noir et assombrit le bord, ce qui ressemble à un vignettage sans en être un. Ce bord est une donnée que la séquence ne possède pas.

CLI : `--stabilise [MODÈLE] --stabilise-method auto|ecc|features --stabilise-reference N --stabilise-min-confidence C`. Les graphies `--stabilize` fonctionnent également.

## Sortie vidéo — `--video`

Tout ce qui précède réduit une vidéo à une seule image. `--video` fait l'inverse : la chaîne est appliquée à chaque image d'une plage, puis réécrite en vidéo. Une seule image est conservée en mémoire à la fois, ce qui permet de traiter des séquences plus longues que la mémoire disponible.

Écarter des images abaisse d'autant la cadence : `--frame-step 5` sur du 25 im/s écrit du 5 im/s, de sorte que le résultat se déroule toujours en temps réel. `--fps` permet de forcer une autre valeur.

**Le codec est une décision de nature légale, pas un détail pratique.** Cette boîte à outils contient des filtres — `ela`, `ghost`, `compression_analysis` — dont le travail est justement de lire l'historique de compression. Écrire une pièce à conviction dans un codec avec perte ajoute une génération de ce qu'ils lisent. Mesuré sur cette version, dix images écrites puis relues :

| Codec | Conteneur | Taille | Aller-retour |
|---|---|---|---|
| **FFV1** | `.avi` | 178 Ko | **exact** |
| RGBA | `.avi` | 211 Ko | exact (non compressé) |
| MJPG | `.avi` | 47 Ko | avec perte |
| XVID | `.avi` | 41 Ko | avec perte |
| mp4v | `.mp4` | 36 Ko | avec perte |

`.avi` utilise donc FFV1 par défaut : sans perte, intra-image, et plus compact que du non compressé. `.mp4` n'offre aucune option sans perte qu'OpenCV écrive de façon fiable ; le demander revient donc à demander un fichier avec perte — et la CLI le signale plutôt que de laisser passer la chose sans commentaire. `avc1`/`H264` sont indisponibles dans cette version et échouent proprement au lieu de produire silencieusement un fichier vide.

Une image dont la taille diffère de la première est une erreur, non un redimensionnement : une chaîne qui redimensionne certaines images et pas d'autres a produit une séquence qui ne veut pas dire une seule chose, et les étirer pour les faire correspondre masquerait précisément cela.

CLI : `--video --video-frames N --frame DÉBUT --frame-step N --fps CADENCE --codec FOURCC`

---

# Catalogue restant

Le docstring de chaque module porte le raisonnement complet ; voici le résumé et les mises en garde les plus importantes.

## Ajuster

**`curves`** — courbe tonale à points de contrôle, interpolée avec une spline monotone (PCHIP) de sorte que la correspondance ne puisse jamais rebrousser chemin et inverser l'ordre tonal comme peut le faire une cubique ordinaire. Préréglages : `linear`, `brighten`, `darken`, `contrast`, `reduce_contrast`, `lift_shadows`, `film`. CLI : `--curves preset=lift_shadows` ou `--curves points=0:0,128:170,255:255`.

**`white_balance`** — chaque méthode automatique suppose quelque chose sur la scène, et échoue quand ce n'est pas vérifié : `gray_world` (la moyenne est neutre — échoue quand une couleur domine), `white_patch` (les pixels les plus clairs sont blancs — échoue sur une zone surexposée), `shades_of_gray` (un compromis, et l'option par défaut). Quand la scène contient un élément connu pour être neutre, `--wb-patch X,Y,W,H` le mesure au lieu de deviner.

**`saturation` / `vibrance`** — la vibrance pondère l'augmentation vers les couleurs peu saturées afin que les couleurs vives ne se figent pas en aplats. À noter que l'atténuation est *proportionnelle* : une couleur à saturation moyenne peut encore gagner plus de saturation brute qu'une couleur presque neutre.

**`color_balance`** — décale les ombres, les tons moyens et les hautes lumières indépendamment, avec des pondérations gaussiennes qui se chevauchent pour que les plages se fondent plutôt que de créer des bandes. Utile quand deux sources lumineuses colorent différemment selon la luminosité. `preserve_luminosity` maintient la luminosité globale fixe pour que seule la couleur change.

**`invert`** — `invert_luminance` inverse la luminosité tout en conservant la teinte, ce qui rend parfois lisible un détail sombre sur fond sombre là où augmenter la luminosité ne fait que le délaver.

## Améliorer

**`nl_means`** — moyenne les patchs qui *se ressemblent* où qu'ils se trouvent, de sorte qu'une texture répétitive se renforce plutôt qu'elle ne se lisse. Le débruiteur le plus lent de la boîte à outils ; le coût croît avec le carré de `search_window`. Régler `h` à partir du bruit mesuré via `estimate_h`, ou utiliser `--nl-means-auto`. `nl_means_denoise_frames` utilise les images voisines comme preuve supplémentaire, sans le flou de mouvement que provoque le simple moyennage d'images sur les objets en mouvement.

**`super_resolution`** — la distinction ici compte plus que partout ailleurs dans la boîte à outils. `upscale` **interpole et n'ajoute aucune information** ; une plaque illisible à la résolution native reste illisible agrandie. `super_resolve` récupère véritablement du détail, car le mouvement infra-pixellaire entre les images échantillonne la scène sur des grilles différentes. Cela nécessite un vrai mouvement infra-pixellaire — `super_resolve_report` indique si une séquence en contient.

La corrélation de phase pilote l'alignement, et elle nécessite du détail à large bande. Une scène fortement périodique (carrelage, briques, une clôture) produit plusieurs pics de corrélation de hauteur similaire, et le décalage mesuré peut être dénué de sens plutôt que simplement imprécis.

**Ce que cela rapporte.** Mesuré face à une vérité terrain, sur des images décalées de quantités connues puis sous-échantillonnées, en comparant la reconstruction ×2 à l'interpolation bicubique d'une seule image :

| Mouvement des images | Gain sur l'interpolation |
|---|---|
| décalages couvrant la grille au demi-pixel | **+1,4 dB** |
| pas entiers uniquement | +0,6 dB |
| faible tremblement aléatoire (±0,3 px) | +1,0 dB |
| fort tremblement aléatoire (±3 px) | –0,1 dB |

Le gain vient de ce que les images échantillonnent *entre* les pixels les unes des autres : avoir du mouvement n'est pas avoir le bon mouvement. La dernière ligne est la mise en garde honnête — une séquence peut satisfaire le critère `usable` de `super_resolve_report`, qui demande seulement si ≥2 images portent un décalage fractionnaire, et ne pas mieux reconstruire qu'un simple agrandissement. Traiter `usable` comme nécessaire et non suffisant, et comparer à `--upscale` avant de se fier au résultat.

> **`--stabilise` et `superres` se contrarient.** Aligner une pile au sous-pixel supprime précisément le mouvement dont la reconstruction se nourrit. La combinaison est mesurée plutôt qu'interdite : une pile stabilisée signale peu de mouvement infra-pixellaire et la CLI avertit. Stabiliser pour *combiner* des images ; ne pas stabiliser pour *reconstruire* à partir d'elles. Une séquence présentant à la fois un fort mouvement de caméra et du détail infra-pixellaire récupérable exigerait un modèle de mouvement complet au sein de la reconstruction, ce que `estimate_shifts` (translation seule) ne fournit pas.

CLI : `--frames 12 --frame-method superres --sr-scale 2 --sr-sharpen 0.6 --sr-max-shift 8`

**`detail_enhancement`** — `local_contrast` est un masque flou à grand rayon, ce qu'est la plupart des curseurs « clarté ». `enhance_detail` préserve les contours, il fait donc ressortir la texture sans halos. `multiscale_detail` accentue les bandes de fréquence indépendamment.

## Corriger

**`perspective`** — rectification à quatre points. Fournir le ratio réel connu de la surface (`KNOWN_RATIOS` : `a4_portrait`, `a4_landscape`, `us_letter`, `credit_card`, `plate_eu`, `plate_us`, `square`), car l'estimer à partir d'une vue en perspective n'est pas fiable. `find_document_corners` détecte automatiquement une surface rectangulaire ; elle renvoie `None` sur une scène encombrée, ce qui est le résultat attendu et non une erreur.

**`fisheye_correction`** — `barrel` utilise le modèle radial polynomial (réglé à la main, convient pour un grand angle modéré) ; `fisheye` utilise le modèle équidistant pour les véritables caméras dôme. Les deux *estiment* la distorsion. Quand la caméra est disponible, calibrer plutôt.

**`aspect_ratio`** — la vidéo SD n'utilise pas de pixels carrés ; affichée sans correction, tout est étiré et toute mesure est fausse dans un axe. `PIXEL_ASPECT_RATIOS` couvre le PAL, le NTSC, le HDV et l'anamorphique. Le mode `pad` de `fit_to_aspect` est le seul qui ne modifie ni la géométrie ni le contenu.

**`undistort`** — la voie la plus défendable : dériver les intrinsèques réelles de la caméra à partir de photographies d'un échiquier, puis inverser exactement cela. `CameraCalibration. is_reliable` vérifie que l'erreur de reprojection est inférieure à un pixel. Une calibration est spécifique à une caméra, un zoom et une mise au point donnés ; appliquer la calibration d'une autre caméra est pire que n'en appliquer aucune, car le résultat paraît plausible tout en étant géométriquement faux.

## Analyser

**`compression_analysis`** — `blockiness_score` compare les sauts d'intensité sur la grille JPEG de 8 pixels à ceux observés ailleurs. `estimate_jpeg_quality` lit directement les tables de quantification d'un JPEG, ce qui est exact plutôt que déduit — mais seulement tant que le fichier reste un JPEG.

La mesure suppose un contenu photographique. Une image dominée par des contours synthétiques durs qui ne tombent pas sur la grille de blocs gonfle le terme intérieur et peut ne montrer aucun blocage quel que soit son historique. Un fort blocage signifie une compression importante, rien de plus ; un réenregistrement uniformise la grille et efface toute différence locale.

## Spécial

**`color_deconvolution`** — sépare des colorants qui se superposent (encre sur imprimé, tampon sur signature) en résolvant dans le domaine de la densité optique, où l'absorption est additive. Au plus trois colorants, puisque trois canaux donnent trois équations, et les colorants aux vecteurs de couleur quasi parallèles se séparent mal. `estimate_stain_vector` mesure un vecteur à partir d'un échantillon d'un seul colorant, ce qui est la façon fiable de construire une séparation pour une encre inconnue.

**`component_separation`** — un détail invisible dans un composite couleur est souvent évident dans une seule composante. Espaces colorimétriques (`rgb`, `hsv`, `hls`, `lab`, `luv`, `ycrcb`, `yuv`, `xyz`), séparation fréquentielle (base contre détail), et plans de bits. La structure dans les plans de bits de poids faible est notable : le bruit naturel du capteur n'en a aucune, donc un motif à cet endroit suggère des données cachées ou une région collée.

**`redaction`** — la seule opération qui ne doit pas être réversible, et les méthodes évidentes échouent. **Le flou est réversible** — c'est une convolution connue, et la déconvolution de Wiener de cette même boîte à outils peut l'annuler. **La pixellisation est réversible pour un texte court à alphabet connu** — rendre chaque plaque candidate et faire correspondre les moyennes de blocs est une attaque documentée et peu coûteuse. Seuls `fill` et `noise` écartent réellement les pixels d'origine ; `fill` est la méthode par défaut et la seule à utiliser pour un document destiné à être publié.

`noise` tire un bruit neuf à chaque exécution sauf si `seed` est fourni : le même préréglage produit alors une image différente à chaque fois. L'occultation est aussi efficace dans les deux cas, mais un résultat qui ne peut pas être reproduit n'est pas une preuve — fixez `seed` lorsque la chaîne doit se rejouer à l'identique. Cela n'affaiblit rien : les pixels d'origine sont détruits quoi qu'il arrive, donc connaître le bruit n'en récupère aucun.

`verify_redaction` corrèle chaque région avec l'original et indique si le contenu a réellement disparu.

**`annotate`** — flèches, formes, texte, et mesure calibrée. `Scale` convertit des pixels en unités ; `measure_distance` (1D), `measure_area` (2D, formule du lacet), `draw_measurement`, `draw_area_measurement` et `draw_scale_bar` les présentent.

Une échelle n'est valide que pour le plan dans lequel elle a été mesurée. Une règle posée au sol calibre les distances au sol et ne dit rien d'un panneau trois mètres plus loin, qui est plus éloigné de la caméra et donc plus petit par pixel. Corriger d'abord la perspective. Les annotations sont dessinées sur une copie — conserver l'original non annoté, car une image annotée est une figure, pas une preuve.

## Mesure — `measure`, `measure_area`, `scale_bar`

Les étapes de chaîne qui enveloppent `annotate`. Un `Scale` ne peut pas transiter par un préréglage JSON ; ces filtres prennent donc **les deux extrémités d'un objet dont la longueur est connue** — une plaque d'immatriculation, une porte, une règle laissée dans la scène — plutôt qu'un rapport pixels/unités qu'il faudrait calculer à la main, l'étape la plus sujette à l'erreur.

| Paramètre | Signification |
|---|---|

| | |
|---|---|
| `point_a`, `point_b` | Ce qui est mesuré (`measure`) |
| `points` | Trois sommets ou plus (`measure_area`), par paires ou en suite de coordonnées |
| `reference_a`, `reference_b` | Les deux extrémités de la référence de longueur connue |
| `reference_length` | La longueur réelle de cette référence, en `unit_name` |
| `unit_name` | Unité de la longueur de référence, `mm` par défaut |

Donner les trois paramètres `reference_a`, `reference_b` et `reference_length`, ou aucun ; une calibration à moitié énoncée est rejetée plutôt que devinée. Sans référence, le libellé est en pixels — honnête, mais rarement ce qu'exige une pièce à conviction. `scale_bar` en exige une : une barre sans calibration est une règle sans unités.

En ligne de commande, la calibration est un *modificateur* partagé par toutes les mesures de la chaîne, car une échelle appartient à un plan de l'image et non à une mesure isolée :

```
cv-tools plaque.jpg \
   --scale-ref 100,200,340,200 --scale-length 520 --scale-unit mm \
   --measure 40,300,290,300 \
   --measure-area 40,320,290,320,290,400,40,400 \
   --scale-bar 200 -o mesure.jpg
```

**Une aire se convertit par le carré de l'échelle linéaire** : une calibration fausse de 5 % rend l'aire fausse d'environ 10 %. Utiliser plutôt `measure_3d` lorsque l'objet mesuré sort du plan calibré — la taille d'une personne ne peut pas se déduire d'une échelle posée sur le sol où elle se tient.

## Annotation — `arrow`, `text`, `shape`

De la présentation, pas de la mesure : ces filtres désignent quelque chose au lecteur sans rien affirmer à son sujet. `shape` dessine un `rectangle`, `circle`, `ellipse`, `line` ou `polygon` ; rectangles, lignes, cercles et ellipses prennent exactement deux points, les polygones trois ou plus. Les points s'écrivent par paires ou en suite de coordonnées : `10,10,60,50` et `[(10, 10), (60, 50)]` désignent la même chose.

```
cv-tools scene.jpg \
   --arrow start=400,300 end=280,210 label=plaque \
   --text text=Piece_A position=20,40 \
   --shape shape=rectangle points=260,190,340,240 -o figure.jpg
```

Étant des étapes de chaîne ordinaires, elles sont consignées dans le préréglage et dans le rapport de traitement, ce qui rend reproductible une pièce annotée.

**`measure_3d`** — la hauteur hors du plan du sol, que l'échelle ci-dessus ne peut pas atteindre. Étant donné l'horizon du plan du sol, le point de fuite des verticales de la scène, et un objet de référence de hauteur connue posé sur ce sol, la hauteur de tout autre objet posé sur le même sol découle d'un rapport anharmonique (cross-ratio) — une quantité que la projection préserve. La méthode est celle de Criminisi, Reid et Zisserman, *Single View Metrology*, IJCV 40(2), 2000.

| Paramètre | Défaut | Signification |
|---|---|---|

| | | |
|---|---|---|
| `base`, `top` | requis | Point de contact au sol et point le plus haut de la cible |
| `reference_base`, `reference_top` | requis | Les deux mêmes points sur l'objet connu |
| `horizon` | requis | Une ligne `y` pour une caméra de niveau, `x1,y1,x2,y2`, ou `a,b,c` |
| `reference_height` | `1800.0` | Hauteur réelle de la référence, dans `unit_name` |
| `vertical_point` | `None` | Point de fuite vertical ; à omettre si les verticales sont parallèles |
| `unit_name` | `'mm'` | Libellé de l'unité |
| `show_horizon` | `True` | Dessine l'horizon sur lequel repose l'estimation |

`measure_height` renvoie deux sensibilités : `uncertainty_per_pixel` pour une erreur d'un pixel sur les points cliqués, et `horizon_uncertainty_per_pixel` pour un décalage d'un pixel de l'horizon. Lisez la seconde d'abord : une base et un sommet se cliquent sur des détails visibles et sont rarement faux de plus d'un ou deux pixels, alors qu'un horizon se *déduit* et se trompe facilement de dix. Les deux sont écrites sous la hauteur, sauf `show_uncertainty=False`.

`horizon_from_lines` évite d'avoir à le deviner : donnez-lui des droites *parallèles dans la scène* — l'arête sol/mur d'un couloir et son arête plafond/mur, les deux bordures d'une route — et elle renvoie l'horizon passant par leur point de fuite. Un jeu de droites suffit si la caméra n'a pas de roulis ; un second jeu dans une autre direction lève cette hypothèse. Le paramètre `horizon` accepte 8 nombres pour deux droites, 16 pour quatre, et **Choisir des points sur l'image** les collecte.

Aucune des deux ne détecte un horizon simplement mal placé : ce sont des pentes locales. Placez l'horizon sur un plafond plutôt qu'au point de fuite et la sensibilité rapportée *diminue*, alors que l'erreur réelle explose. Vérifiez l'horizon sur la scène : il passe par le point où les parallèles convergent.

Des bases de part et d'autre de l'horizon sont refusées : deux objets posés sur un même plan se projettent du même côté de l'horizon de ce plan.

`measure_height` renvoie aussi le nombre sans dessiner, ainsi que `uncertainty_per_pixel` — de combien la réponse varie pour une erreur d'un pixel sur les points cliqués. À lire avant de citer une hauteur ; c'est le plancher de l'erreur, pas l'erreur totale. `vanishing_point` résout un point de fuite à partir de deux lignes parallèles de la scène ou plus, par moindres carrés, et `horizon_from_vanishing_points` construit l'horizon à partir de deux d'entre eux.

L'estimation ne vaut que ce que valent ses hypothèses, et chacune échoue silencieusement :

- **Les deux objets doivent se tenir sur le même plan de sol.** Quelqu'un sur un trottoir, une marche ou une pente est mesuré par rapport à un plan sur lequel il ne se trouve pas.
- **Corriger d'abord la distorsion de l'objectif.** Des lignes droites courbées faussent la géométrie de fuite avant même le début du calcul.
- **La base est le point de contact au sol.** Un espace sous le talon, ou des pieds cachés derrière une voiture, biaise directement le résultat.
- **La précision s'effondre près de l'horizon**, où un seul pixel couvre une distance réelle qui croît rapidement — ce que rapporte justement `uncertainty_per_pixel`.

- **Omettre `vertical_point` suppose que la caméra n'a aucune inclinaison.** Sur une caméra synthétique à 2,5 m de hauteur avec une référence de 1,8 m, cette hypothèse a surestimé d'environ 16 mm pour 5° d'inclinaison et de 33 mm pour 18°. La plupart des caméras de vidéosurveillance sont inclinées, donc fournir le point de fuite vertical.

---

# Rapports d'analyse (pas des étapes de chaîne)

Les mesures ci-dessus qui renvoient des chiffres plutôt qu'une image sont enregistrées ensemble dans `ANALYSIS_REGISTRY`, à côté du registre des filtres. Elles n'entrent jamais dans une chaîne : elles décrivent l'indice au lieu de le modifier, et en exécuter une laisse le pipeline intact.

| Nom dans le registre | Fonction | Module | Lit | CLI |
|---|---|---|---|---|
| `noise` | `noise_report` | `cv_tools.filters.noise_analysis` | les pixels | `--noise-stats` |
| `ela` | `ela_stats` | `cv_tools.filters.ela` | les pixels | `--ela-stats [QUALITÉ]` |
| `clone` | `detect_copy_move` | `cv_tools.filters.clone_detection` | les pixels | `--clone-stats` |
| `compression` | `compression_report` | `cv_tools.filters.compression_analysis` | pixels + fichier | `--compression-stats` |
| `ghost` | `ghost_report` | `cv_tools.filters.jpeg_ghost` | les pixels | `--ghost-stats` |
| `metadata` | `metadata_report` | `cv_tools.filters.metadata_forensics` | le fichier | `--metadata-stats` |

Chaque entrée porte la présentation de son propre rapport — une ligne d'en-tête, ses lignes de contenu et la mise en garde qui le referme — si bien que la CLI affiche exactement ce que montrent l'onglet **Analysis** de l'interface graphique et celui du tableau de bord web. Les options de la dernière colonne sont engendrées à partir des mêmes entrées : c'est pourquoi un rapport ajouté au registre apparaît dans les trois interfaces sans qu'aucune ne soit modifiée. `--list-analyses` affiche l'ensemble enregistré.

| Rapport | Paramètres |
|---|---|
| `noise` | `block_size=32` |
| `ela` | `quality=90`, `block_size=16` |
| `clone` | `block_size=16`, `step=1`, `coefficients=4`, `quantization=4.0`, `min_distance=0.0`, `min_matches=8`, `min_variance=12.0`, `search_window=3`, `max_blocks=300000` |
| `compression` | `block_size=32` (plus le chemin du fichier source) |
| `ghost` | `qualities=(50…100 par pas de 5)`, `block_size=16` |
| `metadata` | aucun |

**`compression` et `metadata` lisent le conteneur, pas le résultat de la chaîne.** Les tables de quantification et l'EXIF résident dans le fichier sur le disque : ces deux rapports décrivent donc l'image telle qu'elle a été ouverte, quel que soit le nombre de filtres appliqués depuis — et ils n'ont rien à lire du tout si l'image ne provient pas d'un fichier. L'interface graphique le signale au lieu d'échouer ; le tableau de bord écrit le fichier téléversé par le navigateur dans

une copie temporaire portant son propre nom, pour que le rapport cite toujours le nom de fichier que vous reconnaissez.

Chaque ligne porte une gravité : `flag` pour un constat qui mérite examen, `info` pour un constat qui mérite d'être connu, et rien pour une simple mesure. **Aucune des trois n'est une conclusion.** Chaque rapport se termine par une note indiquant ce que la mesure ne peut pas dire, et ces notes sont la forme courte des mises en garde détaillées sous chaque filtre ci-dessus.

```
from cv_tools.filters import report_lines, resolve_analysis, run_analysis

spec = resolve_analysis('ghost')
report = run_analysis(spec, image=pipeline.current, params={'block_size': 8})
print('\n'.join(report_lines(spec, report)))     # ce qu'affiche la CLI

report['outlier_count']                          # ou lire le dict directement
```

`run_analysis` fournit l'image et/ou le chemin selon ce que le rapport demande, et lève `ValueError` au lieu de deviner lorsqu'il en manque un. `render_report` renvoie les mêmes lignes sous forme d'objets `Row(label, value, severity, indent)`, pour une interface qui les colore.

---

# Fonctions d'aide ROI (pas des étapes de chaîne)

`analyze_roi(image, roi)` renvoie la moyenne, l'écart-type, le min et le max par canal, plus le nombre de pixels. Exposé sur la CLI comme `--analyze-roi X,Y,W,H`, qui s'exécute sur l'image **traitée**.

`apply_to_roi(image, roi, filter_fn, **kwargs)` applique n'importe quel filtre à une région seulement, en laissant le reste de l'image inchangé.

`get_centered_roi(shape, w, h)` et `roi_from_ratio(shape, x, y, w, h)` construisent des régions sans coder en dur des coordonnées en pixels.